

Reconocimiento de caracteres G, H e I mediante un perceptrón multicategoría

Preparación de imágenes, entrenamiento, evaluación y análisis de mapas de calor

Introducción

En este trabajo se desarrolló un sistema sencillo de reconocimiento de caracteres para clasificar imágenes correspondientes a las letras G, H e I. El modelo utilizado es un perceptrón multicategoría con tres neuronas de salida, una por cada letra.

Las imágenes se cargaron desde carpetas organizadas por clase, se transformaron a escala de grises y se convirtieron en tensores. Posteriormente, cada imagen se representó como un vector de 64 píxeles y se transformó a valores bipolares para realizar el entrenamiento.

1. Objetivo y herramientas utilizadas

El objetivo fue entrenar un modelo capaz de distinguir entre las letras G, H e I, medir su efectividad sobre un conjunto de prueba y analizar los patrones aprendidos mediante mapas de calor.

Herramienta	Uso dentro del proyecto
Python	Lenguaje utilizado para implementar el procesamiento y el perceptrón.
PyTorch	Manejo de tensores, pesos y operaciones matriciales.
torchvision.ImageFolder	Carga automática de imágenes organizadas en carpetas G, H e I.
DataLoader	Recorrido de los conjuntos de entrenamiento y prueba con lotes de una imagen.
Matplotlib	Visualización de caracteres, mapas de calor y resultados.

2. Organización y preparación de los datos

El notebook cargó tres clases en el orden G, H e I. El conjunto de entrenamiento contiene 30 imágenes y el conjunto de prueba contiene otras 30 imágenes.

Clase	Índice	Salida objetivo bipolar
G	0	[1, -1, -1]
H	1	[-1, 1, -1]
I	2	[-1, -1, 1]

Cada imagen se convirtió a escala de grises y a un tensor. Después, los valores de los píxeles se binarizaron con un umbral de 0.5 y se transformaron a la representación bipolar.

$$x_b = 2 \cdot I(x > 0.5) - 1$$

Con esta operación, los píxeles del fondo y del carácter quedan expresados mediante -1 y 1. También se añadió una entrada constante igual a 1 para representar el bias.

Fragmento de preparación de imágenes

```
transform_train = transforms.Compose([
    transforms.Grayscale(),
    transforms.ToTensor()
])

train_dataset = ImageFolder(train_directory, transform_train)
test_dataset = ImageFolder(test_directory, transform_test)
```

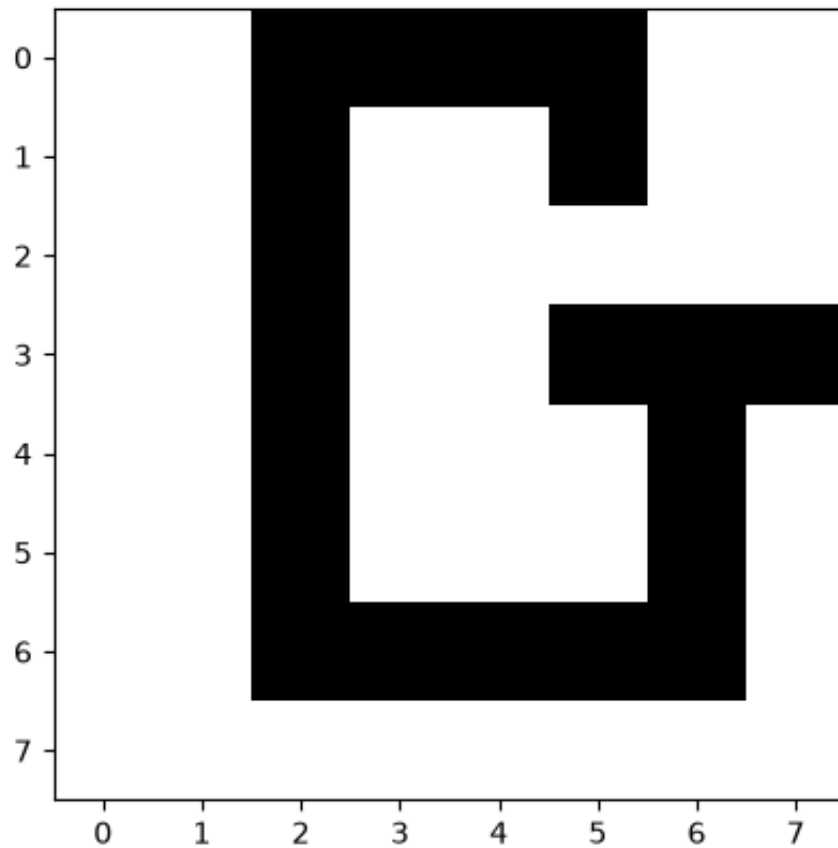


Figura 1. Ejemplo de una imagen de entrenamiento en formato de 8×8 píxeles.

3. Funcionamiento del perceptrón

El modelo se creó con 64 entradas, correspondientes a los 64 píxeles de una imagen de 8×8 , y tres salidas. Cada neurona posee su propio vector de pesos y calcula una puntuación para una clase.

$$y_{in}^{(k)} = x \cdot w_k$$

La función de activación utiliza un umbral simétrico. La salida de cada neurona puede ser 1, -1 o 0.

$$y_k = 1 \text{ si } y_{in}^{(k)} > \theta; \quad y_k = -1 \text{ si } y_{in}^{(k)} < -\theta; \quad y_k = 0 \text{ en otro caso}$$

Durante el entrenamiento se empleó una tasa de aprendizaje de 0.01 y el umbral predeterminado de 1. Cuando una salida no coincidió con su objetivo, se actualizaron los pesos.

$$w_k(\text{nuevo}) = w_k(\text{anterior}) + \alpha \cdot t_k \cdot x$$

$$b_k(\text{nuevo}) = b_k(\text{anterior}) + \alpha \cdot t_k$$

Fragmento de actualización del perceptrón

```
if y[i].item() != t[i].item():
    self.weights[i][1:] += learning_rate * x_i.T[1:] * t[i]
    self.weights[i][0] += learning_rate * t[i]
```

Flujo general del sistema de reconocimiento

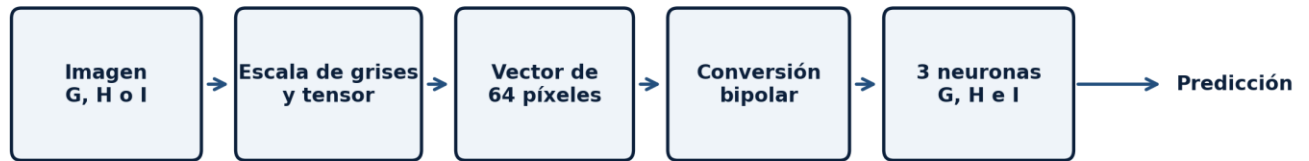


Figura 2. Flujo general del reconocimiento desde la imagen hasta la clase predicha.

4. Entrenamiento del modelo

El perceptrón se entrenó con un tamaño de lote igual a 1 y con las imágenes de entrenamiento mezcladas en cada recorrido. El proceso imprimió épocas numeradas de 0 a 12, lo que corresponde a 13 recorridos antes de alcanzar la condición de parada.

Parámetro	Valor utilizado
Entradas	64 píxeles + 1 bias
Salidas	3 neuronas: G, H e I
Tasa de aprendizaje	0.01
Umbral	1
Recorridos observados	13, numerados de 0 a 12

5. Resultados de la clasificación

5.1 Exactitud general

El conjunto de prueba contiene 30 imágenes. El modelo clasificó correctamente 27 y cometió 3 errores.

$$\text{Exactitud} = (27 / 30) \times 100 = 90 \%$$

Letra real	Correctas	Total	Exactitud
G	10	10	100 %
H	8	10	80 %
I	9	10	90 %
Total	27	30	90 %

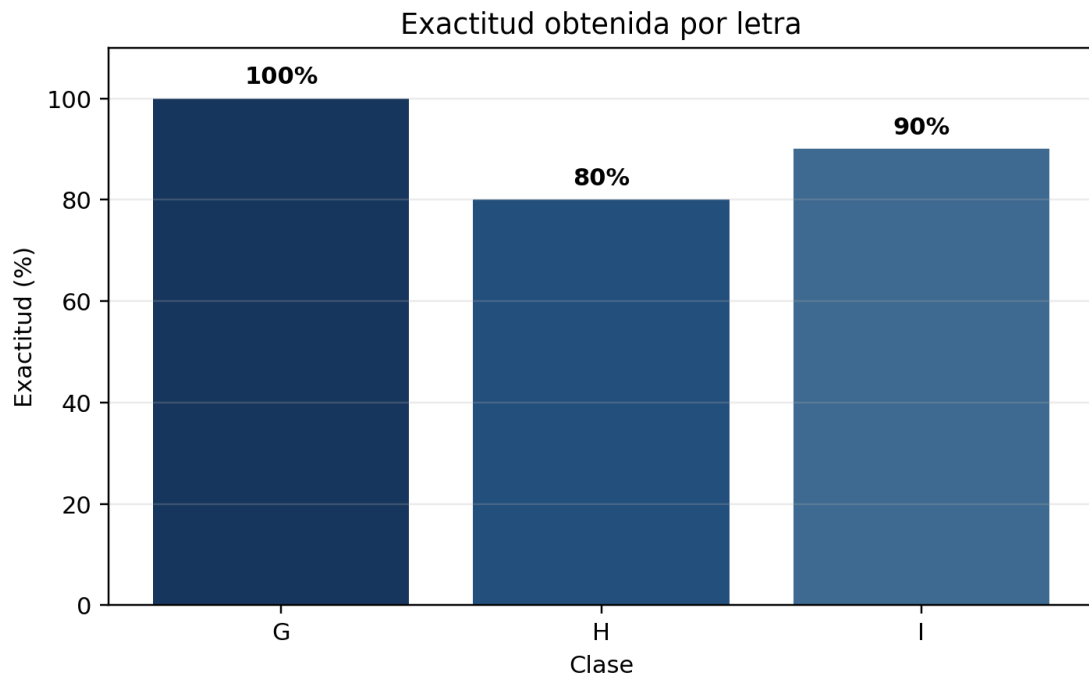


Figura 3. Exactitud obtenida para cada letra del conjunto de prueba.

5.2 Matriz de confusión

La matriz de confusión muestra que todas las imágenes de G fueron clasificadas correctamente. Dos imágenes de H y una imagen de I fueron clasificadas como G.

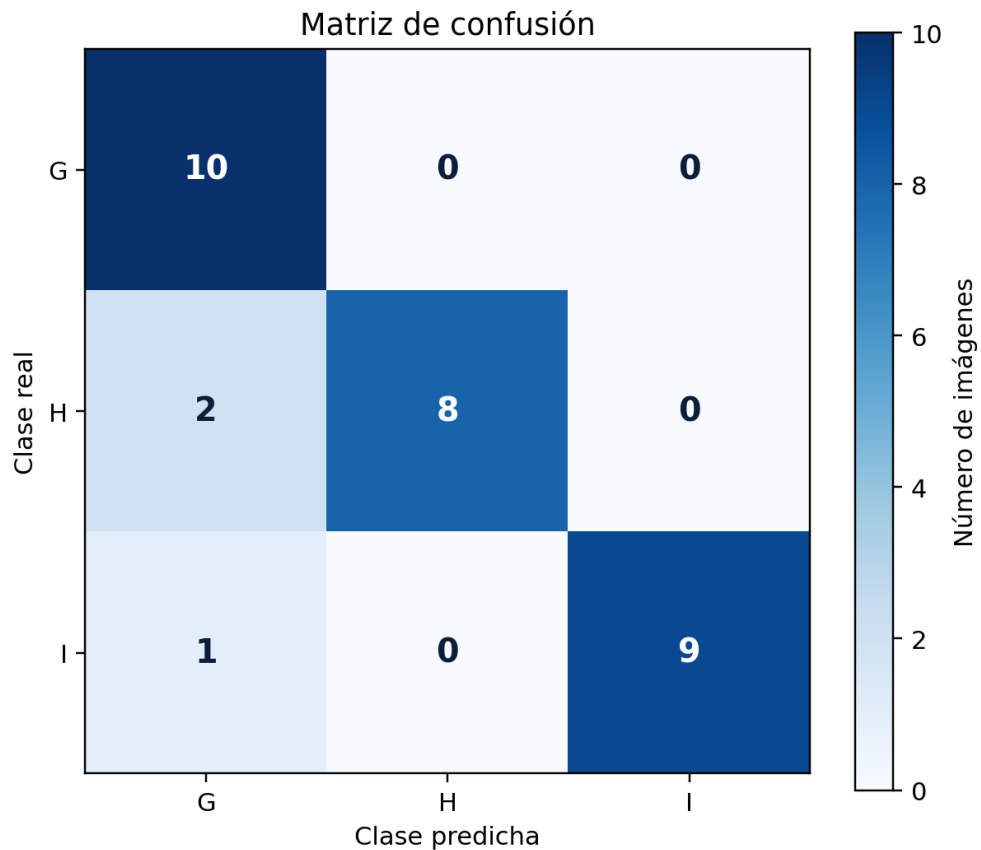


Figura 4. Matriz de confusión del modelo para las clases G, H e I.

5.3 Ejemplo de una predicción incorrecta

En la ejecución mostrada por el notebook se seleccionó una imagen real de la letra I. Las tres neuronas produjeron la salida [0, 0, 0]. Como `torch.argmax` devuelve el primer índice cuando existe un empate, la predicción final fue G.

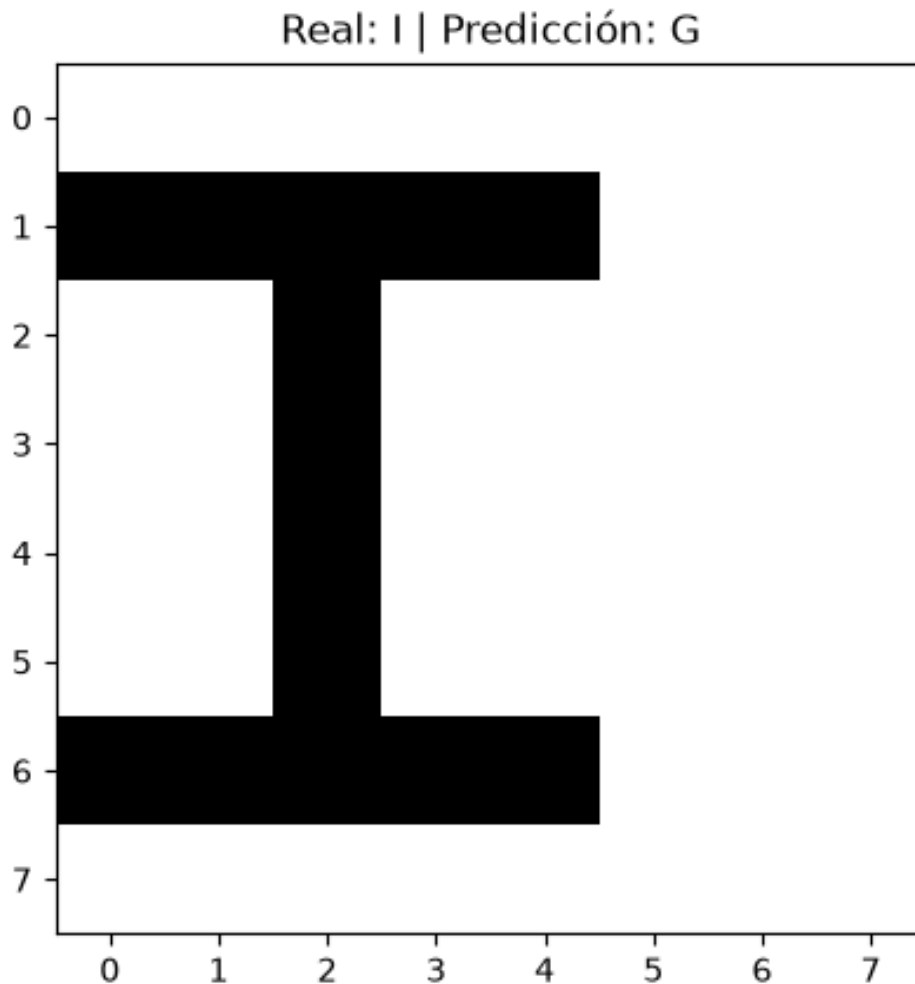


Figura 5. Ejemplo real de la letra I clasificado como G debido a un empate entre las salidas.

6. Mapas de calor de los pesos aprendidos

Los pesos de cada neurona se reorganizaron en una matriz de 8×8 y se normalizaron entre 0 y 1. Las zonas más claras indican posiciones con mayor influencia positiva dentro del patrón aprendido; las zonas oscuras representan una influencia menor.

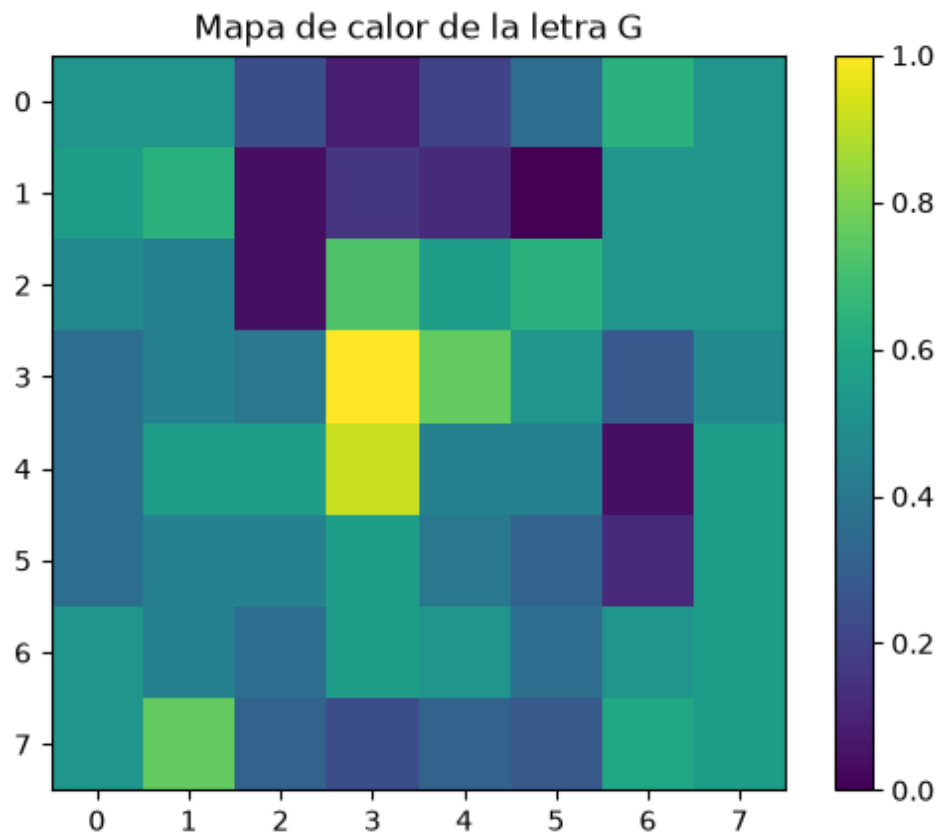


Figura 6. Mapa de calor de los pesos correspondientes a la letra G.

En la letra G se observan valores elevados principalmente en la región central y en algunas zonas inferiores. El patrón no forma una plantilla exacta de la letra, sino una representación de las posiciones que más influyeron en la clasificación.

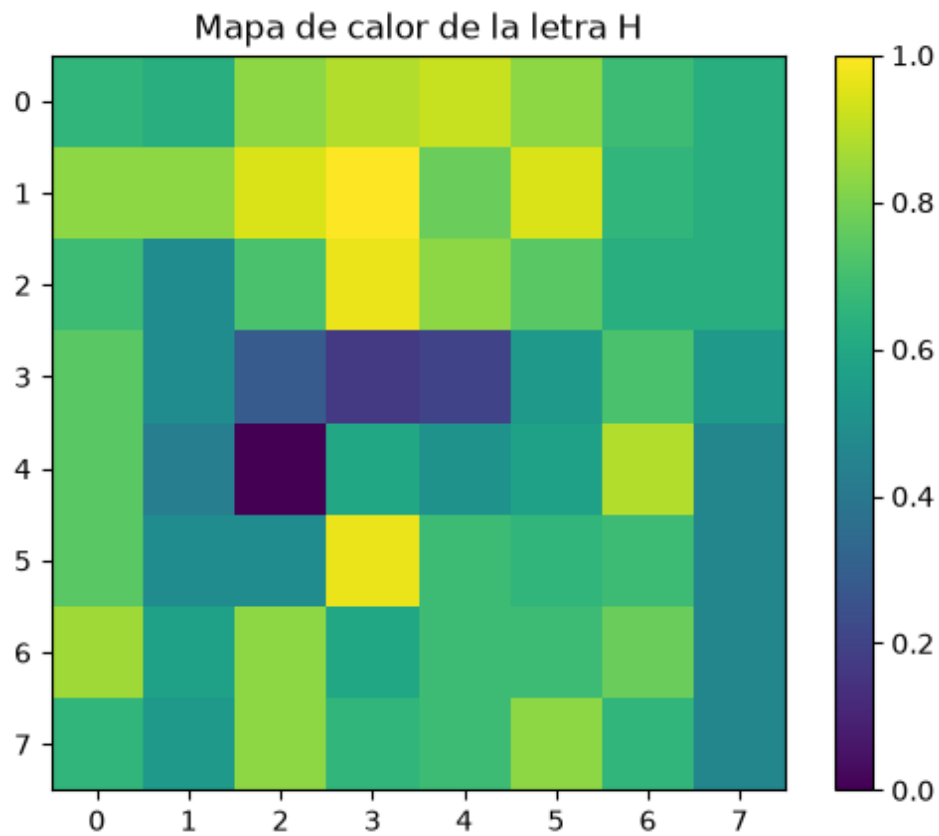


Figura 7. Mapa de calor de los pesos correspondientes a la letra H.

El mapa de H presenta activaciones distribuidas en gran parte de la matriz. La menor exactitud de esta clase, 80 %, indica que algunos de sus patrones se aproximaron más a la decisión de G.

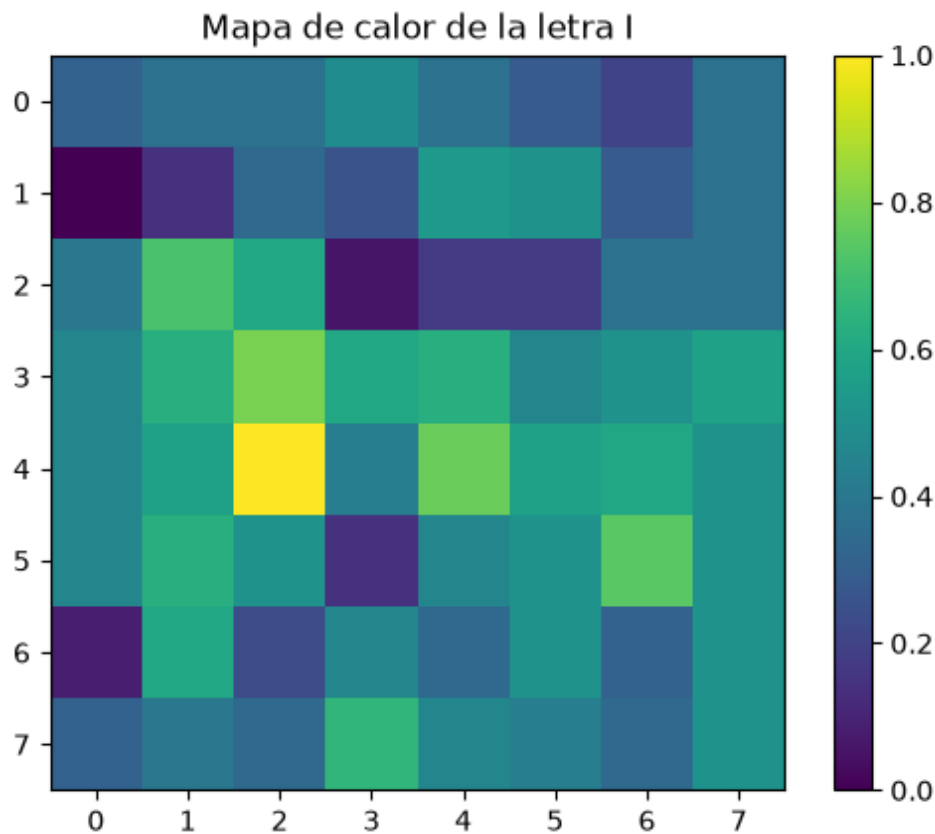


Figura 8. Mapa de calor de los pesos correspondientes a la letra I.

Para I se aprecia una concentración destacada en la zona central. La clase logró 90 % de exactitud y solamente una imagen fue confundida con G.

7. Análisis de los resultados

- El modelo reconoció perfectamente la letra G, con 10 aciertos de 10.
- La letra H fue la clase más difícil: 2 de sus 10 imágenes fueron clasificadas como G.
- La letra I obtuvo 9 aciertos y presentó un caso ambiguo cuya salida fue [0, 0, 0].
- Los tres errores terminaron en la clase G, lo que sugiere que la neurona de G domina algunos patrones ambiguos.
- La exactitud de 90 % es alta para un perceptrón sencillo, aunque todavía existe margen de mejora en la diferenciación de H e I.
- Los mapas de calor ayudan a interpretar qué posiciones de los 64 píxeles tuvieron mayor peso en cada neurona.

Una posible mejora consistiría en aumentar la variedad de imágenes, aplicar transformaciones controladas durante el entrenamiento o utilizar un criterio explícito para resolver empates, en lugar de aceptar automáticamente el primer índice producido por argmax.

Conclusión

El ejercicio permitió implementar un sistema de reconocimiento de las letras G, H e I mediante un perceptrón multicategoría. Las imágenes se procesaron en escala de grises, se convirtieron a tensores y se representaron mediante valores bipolares. El modelo utilizó 64 entradas, tres neuronas de salida, una tasa de aprendizaje de 0.01 y un umbral de 1.

Después de 13 recorridos de entrenamiento, el sistema alcanzó una exactitud general de 90 %, con 27 predicciones correctas de 30. La letra G obtuvo 100 %, H obtuvo 80 % e I alcanzó 90 %. Los errores se

concentraron en predicciones hacia G, especialmente en casos ambiguos. Los mapas de calor permitieron visualizar la distribución de los pesos aprendidos y complementaron el análisis del comportamiento del modelo.